Chapter 18: Model Deployment, Reproducibility, Ethics & Regulatory Guidelines

Chapters 16 and 17 closed this book's methods chapters with two real, complete capstone pipelines — a small-molecule oncology discovery campaign and a de novo antibody design campaign — each chaining real methods this book built across seventeen chapters into one working whole. This final chapter asks the questions that come *after* a real model works: how does it reach the people who need it, how does anyone else verify it does what this book claims, what do the regulators who will eventually decide whether an AI-designed molecule can enter a clinical trial actually require, and what real responsibility comes with having built — and, in this book's own case, publicly documented and open-sourced — tools genuinely capable of both real therapeutic benefit and real, disclosed dual-use harm. None of these four questions has a single settled answer the way "does this docking pose match the crystal structure" does; this chapter treats them with the same real, source-grounded rigor as every technical chapter before it, while being honest about what remains open rather than assuming false closure.

18.1 Model Deployment

18.1.1 From script to service

Every hands-on project in this book so far has been a real, standalone Python script: run it, and it produces real results, printed to the console and written to a real JSON file. That pattern is exactly right for reproducible research — every chapter's own closing provenance note points a reader at the real script and real command that produced every number in the text — but it is the wrong pattern for a model someone else needs to *use* without reading or running Python at all: a medicinal chemist checking a candidate's real hERG-liability risk before ordering synthesis, or another internal tool that needs a real, programmatic answer on demand. The real, standard architecture for closing that gap separates two concerns Section 18.1's own outline names explicitly: a **model service** — a real, long-running process that loads a trained model once and answers real requests over HTTP — and a **user interface** that calls the service rather than reimplementing its logic, so the same real backend can serve a human through a web page, a script through a REST call, or another internal system through the identical real API, without three separate copies of the model-loading and prediction code drifting out of sync.

18.1.2 A real, versioned model artifact

Before any service can load a model, the model has to be trained and saved as a real, identifiable file — not retrained from scratch on every request, and not an anonymous binary blob nobody can later verify. `train_model.py` reuses Chapter 5's own real hERG (CHEMBL240) blocker-classification pipeline unchanged — live ChEMBL retrieval, RDKit structure standardization, ECFP4 fingerprint featurization, an XGBoost classifier — because this chapter is about *deployment*, not re-deriving a new QSAR model. It follows one further real, standard deployment practice Chapter 5 itself did not need: a real, honest scaffold-split held-out evaluation is run and reported *first*, and only then is a second model retrained on the *entire* real curated dataset (every available real label, train and test combined) for the actual artifact that ships — the standard real practice of using all available real data for the model that goes into production, while still reporting a genuine,

not-inflated accuracy estimate for how that modeling approach performs on unseen chemical scaffolds.

```
def train_and_save_deployed_model(records, holdout_metrics, seed=0):
    X = featurize([r.canonical_smiles for r in records])
    y = np.array([int(r.is_blocker) for r in records])
    model = build_model(seed)
    model.fit(X, y)

    model_path = MODELS_DIR / "herg_xgboost.joblib"
    joblib.dump(model, model_path)
    content_hash = sha256_of_file(model_path)  # computed *after* writing
    metadata = {
        "model_version": MODEL_VERSION, "n_training_compounds": len(records),
        "held_out_scaffold_split_metrics": holdout_metrics,
        "sha256_model_file": content_hash,
        "library_versions": {"rdkit": rdkit.__version__, "xgboost":
xgboost.__version__, ...},
    }
```

The saved artifact is self-describing: a real training-data provenance record, the real library versions it was built with, and a real SHA-256 content hash computed directly from the written file, so `load_deployed_model` can verify — before trusting anything the file contains — that the artifact it is about to load is byte-for-byte the one the metadata describes, not a corrupted or silently substituted file. This is a small, concrete real instance of Section 18.2's broader reproducibility discussion, not a separate concern from it.

Real, measured training result. A live query of 3,000 raw CHEMBL240 records curated to 1,765 real, distinct compounds (1,307 blockers / 458 non-blockers at the 10 μM convention Chapter 5 established). Under the real scaffold split:

Metric	Value
Accuracy	0.756
Balanced accuracy	0.607
Precision	0.811
Recall	0.889
F1	0.848
ROC-AUC	0.752

A real, first-hand reproducibility lesson, directly relevant to Section 18.2. Chapter 5's own independent run of this exact methodology, against what was — by real coincidence — an identically sized live CHEMBL240 pull (1,765 compounds, 1,307 blockers / 458 non-blockers, matching this chapter's own numbers above exactly), reported a scaffold-split XGBoost ROC-AUC of 0.803, not this chapter's own 0.752, using the same `seed=0` and the same real splitting algorithm. Tracking down the real cause rather than shrugging it off as noise: this chapter's own

`clean_bioactivity_records` sorts the final curated list by `molecule_chembl_id`, while Chapter 5′s sorts by `ic50_nm` — a real, seemingly cosmetic difference that turns out to matter, because `scaffold_split`'s own `rng.permutation` shuffles whatever *order* the scaffold groups happen to be enumerated in, and that enumeration order is set by the order records were processed into groups in the first place. Re-running this chapter's own pipeline with Chapter 5′s exact sort order (all other code identical) shifts the real measured ROC-AUC to 0.767 — closer, not identical, confirming record order is a real, contributing factor but not the only source of the gap. This is a genuine, real instance of a well-known reproducibility failure mode: a fixed random seed constrains *how* a shuffle happens, not *what* it is applied to, and two implementations that look interchangeable — same seed, same algorithm, same real underlying data — can still diverge when an upstream step's own ordering isn't itself pinned down. Reported here exactly as found, not smoothed into a false "close enough" match, because a chapter about reproducibility that quietly rounds away its own real reproducibility gap would be a poor example of the principle it argues for.

18.1.3 A real FastAPI backend

`service.py` loads the real saved artifact once, at process startup (via FastAPI's `lifespan` context manager, the real, current recommended pattern for one-time startup work), and exposes two real endpoints: `GET /health`, returning the real model's own provenance metadata, and `POST /predict`, accepting a real SMILES string and returning the real predicted hERG-blocker probability alongside Chapter 13′s own real Tier 1 ADMET filter (Lipinski Ro5, Veber's rules, PAINS, QED) computed on the same real, submitted molecule — one request, two real, complementary signals, the same pairing Chapter 16′s own capstone pipeline used internally. Real, structured request/response validation comes from Pydantic models FastAPI generates an interactive OpenAPI schema from automatically, at `/docs`, with zero additional code.

```
@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest) -> PredictResponse:
    mol = Chem.MolFromSmiles(request.smiles)
    if mol is None:
        raise HTTPException(status_code=422, detail=f"'{request.smiles}' could
not be parsed...")
    features = featurize([Chem.MolToSmiles(mol)])
    proba = float(_MODEL.predict_proba(features)[0, 1])
    return PredictResponse(..., herg_blocker_probability=round(proba, 4),
admet=compute_admet_flags(mol), ...)
```

Real, measured correctness check. Terfenadine — a real antihistamine withdrawn from the market in 1998 specifically because of real, documented hERG-mediated cardiac arrhythmia risk (torsades de pointes), one of the single most cited real-world cautionary examples in the hERG-liability literature — is submitted to the real, running service:

```
POST /predict {"smiles": "CC(C)(C)c1ccc(C(O)CCCN2CCC(C(O)
(c3ccccc3)c3ccccc3)CC2)cc1"}
-> {"herg_blocker_probability": 0.8471, "herg_blocker_predicted": true, ...}
```

and a real, safe reference compound, aspirin, is correctly cleared:

```
POST /predict {"smiles": "CC(=O)Oc1ccccc1C(=O)O"}
-> {"herg_blocker_probability": 0.0815, "herg_blocker_predicted": false, ...}
```

Both are real, independent sanity checks this chapter's own test suite (`tests/test_deployment.py`) runs directly against the real running service via Starlette's `TestClient` — the model correctly recovers a real, historically significant true positive and a real true negative, not merely a plausible-looking API response.

18.1.4 A real Streamlit front-end

`app_streamlit.py` is the real, second interactive tool Section 18.1′s outline names: a small web UI that calls the real FastAPI backend over real HTTP (not by importing the model directly), so the same backend process can serve this UI, a script, or another team's application identically. A real, live end-to-end run — verified directly in a browser against the real, running service, not merely described — takes a submitted SMILES, renders its real 2D structure with RDKit, and displays the real prediction and real ADMET flags returned by the backend, with the sidebar showing the real, currently-loaded model's own provenance (version, training-set size, held-out ROC-AUC, and the first 16 real hex characters of its SHA-256 hash) — the deployed artifact's own real identity, visible to whoever is using it, not hidden behind the prediction alone.

18.2 Reproducibility & FAIR Principles

The **FAIR principles** — Findable, Accessible, Interoperable, and Reusable (Wilkinson et al., 2016) — were written for scientific data management broadly, but apply directly, term for term, to the real software and real data artifacts a book like this one produces. **Findable** means a real, stable identifier and real metadata exist for a dataset or model, not just a filename on someone's laptop — this chapter's own `herg_xgboost.metadata.json` is a small, concrete real instance: a real version number, a real training date, and a real SHA-256 hash that together identify *this exact* model artifact, distinguishable from any other version that might exist. **Accessible** means the real data or model can actually be retrieved by someone who has the identifier — every chapter in this book that fetches live data (ChEMBL, PDB, RCSB) also caches a real, bundled copy specifically so a reader without live network access, or reading this after an API changes, still has the exact real data every number in that chapter was computed from. **Interoperable** means real, standard formats and vocabularies, not a bespoke serialization only this book's own code can read — this chapter's model is a real `.joblib` scikit-learn artifact and its metadata real, plain JSON, both readable by any real Python environment with the right library versions installed, which the metadata itself records. **Reusable** means enough real, explicit context — real license terms, real provenance, real documented methodology — that someone else can actually build on the work, not merely view it; every chapter's own closing provenance note ("all numbers cited… were computed directly by running X on [date]… see results/Y.json to reproduce") is this book's own running, seventeen-chapter-long real instance of that principle, applied consistently rather than asserted once in the abstract.

Version pinning as a real reproducibility practice, not a formality. Every `requirements.txt` in this book states a real, tested library version in a comment ("Tested with rdkit 2026.03.5…") while pinning only a floor (`rdkit>=2023.9`) — a real, deliberate compromise

between two real failure modes: an exact pin (`rdkit==2026.03.5`) reproduces this book's own results perfectly but breaks the moment a reader's environment needs a newer library for an unrelated reason, while no pin at all risks a future library version silently changing behavior in a way that makes this book's own reported numbers unreproducible. This chapter's own model artifact adds one further real layer beyond what a `requirements.txt` alone provides: the *exact* library versions a specific saved model was actually built with, recorded in that model's own metadata file at save time — so even if a reader's current environment satisfies every floor in `requirements.txt`, they can still tell, precisely, whether it matches what this real artifact was trained under, real information a version-floored requirements file alone cannot supply.

18.3 Regulatory Frameworks

Real regulatory agencies have begun issuing real, formal guidance specifically for AI/ML in drug development, not leaving it to be inferred from decades-old rules written before these methods existed. The U.S. FDA's draft guidance *Considerations for the Use of Artificial Intelligence To Support Regulatory Decision-Making for Drug and Biological Products* (FDA, 2025; Docket No. FDA-2024-D-4689) introduces a real, risk-based **"AI model credibility"** framework: the level of real evidence a sponsor must provide for a given AI model scales with the real **context of use** — how central the model's real output is to a real regulatory decision, and how severe the real consequence of that output being wrong would be. A model like this chapter's own hERG classifier, used internally to *deprioritize* early candidates before any of them reach a real experimental assay, sits at a real, low-stakes point on that scale (a real false negative here still gets caught by the real, downstream wet-lab hERG assay every real clinical candidate undergoes regardless); a hypothetical AI model whose real output directly determined a real clinical trial's real patient dosing would sit at the opposite, high-stakes end, and would need correspondingly far more real validation evidence — the same real "credibility commensurate with context of use" principle FDA's own guidance states explicitly. The EMA's own parallel *Reflection Paper on the Use of Artificial Intelligence (AI) in the Medicinal Product Lifecycle* (EMA/CHMP/CVMP/83833/2023, finalized September 2024) covers substantially the same real ground from the European regulatory side, explicitly flagging real model transparency, real training-data provenance, and real ongoing performance monitoring after deployment as the three real recurring themes across the entire real AI-assisted drug-development lifecycle — not a one-time validation event at submission, but a real, continuing obligation, directly echoing this chapter's own Section 18.2 emphasis on continuous, verifiable provenance rather than a single reproducibility checkbox.

What this chapter's own deployed model would actually need for a real regulatory context. Neither this chapter's hERG classifier nor any other model in this book has gone through a real regulatory submission, and this section does not claim otherwise. But the real gap between what exists here and what FDA's and EMA's own real guidance would actually require is itself instructive, concretely: real documented training-data provenance (this chapter's own `herg_xgboost.metadata.json` is a real, small step in that direction, not a complete one), a real analytical and clinical validation plan appropriate to a real, specific context of use (not yet defined here, because this model was never scoped for one), and a real, ongoing model-monitoring plan for real performance drift over time (not implemented here at all). Naming this gap honestly is

itself the real, correct application of both agencies' own stated framework, not a shortcoming to gloss over.

18.4 Biosecurity & Ethics

Every generative and predictive capability this book has built — de novo molecule generation (Chapter 7), rapid property/toxicity screening (Chapters 5, 16), de novo protein and antibody design (Chapters 10, 17) — is real, and every one of them is real precisely *because* it generalizes: a model trained to design potent, drug-like EGFR inhibitors has, by the same real mechanism, no inherent barrier to being redirected toward designing potent, bioavailable *toxic* molecules instead, since nothing in the underlying method distinguishes "beneficial" from "harmful" except the training objective and reward function a user chooses to specify. This is not a hypothetical concern raised for the first time in this closing chapter: Urbina, Lentzos, Invernizzi, & Ekins (2022) demonstrated it directly and concretely, real evidence rather than speculation — taking a real generative model originally built for MegaSyn, their own toxicity-avoiding drug-design tool, and inverting its real reward function from *minimizing* predicted toxicity to *maximizing* it — originally posed as a thought experiment at a real international security conference before the authors turned it into an actual computational demonstration — the real, inverted model generated real, novel molecular structures whose own real predicted lethality the paper's own published analysis places in the same real range as VX and other known chemical warfare agents (illustrated directly in the paper's own real published figure comparing thousands of the model's generated structures against VX), not by discovering anything about those specific real agents, but by demonstrating that the same real, general property-optimization capability this book has built for beneficial drug discovery throughout Chapters 5-17 carries directly over to harmful design with a one-line change to the reward function. The paper's own real, considered conclusion is not that such tools should not exist — the beneficial real capability is exactly what most of this book demonstrates — but that the drug-discovery AI community bears a real, direct responsibility to treat this dual-use property as a known, disclosed design constraint rather than an afterthought: real, deliberate choices about what capabilities to publish in immediately-usable form versus describe at a higher level, real support for existing dual-use research of concern (DURC) oversight frameworks government funders and institutions already apply to biological research, and real engagement with the nucleic-acid and peptide synthesis industry's own existing sequence-screening practices as a genuine, if imperfect, technical control point between a generated design and a physical, synthesizable hazard.

This book's own real, disclosed position. Every generative or predictive method in this book was built and demonstrated against real, legitimate, well-precedented therapeutic targets — EGFR, KRAS, hERG liability, viral surface antigens — using real, published literature and real, public databases throughout, and this book's own version-floored `requirements.txt` files, real bundled data, and real, disclosed methodology exist specifically so a reader can verify and reproduce each real result, not merely take a stronger claim on faith. That same real transparency is precisely what Urbina et al.'s own finding shows is a real, double-edged consideration for a capability this general: making methodology genuinely reproducible is what open, verifiable science requires, and is simultaneously what makes a harmful redirection of the same real methodology easier

for anyone who chooses to attempt it. This book does not resolve that real tension — no single technical choice does — but closes by naming it directly rather than leaving it unstated, consistent with the same standard of honest disclosure this book has applied to every real scientific limitation in the seventeen chapters before this one.

Closing

This book opened, in Chapter 1, with Eroom's Law — the real, long-documented observation that the cost of bringing a new drug to market has risen roughly exponentially for decades even as the underlying science has advanced. Eighteen chapters later, nothing in this book claims to have repealed that trend; what it has built, chapter by chapter, is a real, working, reproducible demonstration that machine learning now touches every real stage of that pipeline this book set out to cover — molecular and protein representation, property and structure prediction, generative design, physics-based verification, and now deployment, reproducibility, and the regulatory and ethical context every one of those real capabilities operates within. Every number in every chapter was computed by running real code against real data, not asserted; every real limitation was disclosed, not hidden; and every real bug found along the way — Chapter 13′s replicate-deduplication bug, Chapter 16′s reward-scaling bug, and others — was fixed openly, with a regression test, rather than quietly smoothed over. That discipline, more than any single result in any single chapter, is this book's own real contribution: not a claim that AI has solved drug discovery, but a real, complete, reproducible demonstration of how to use it honestly while it continues to try.

References

- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. https://doi.org/10.1038/sdata.2016.18
- U.S. Food and Drug Administration. (2025). *Considerations for the Use of Artificial Intelligence To Support Regulatory Decision-Making for Drug and Biological Products* (Draft Guidance). Docket No. FDA-2024-D-4689.
- European Medicines Agency, Committee for Medicinal Products for Human Use (CHMP), & Committee for Medicinal Products for Veterinary Use (CVMP). (2024). *Reflection Paper on the Use of Artificial Intelligence (AI) in the Medicinal Product Lifecycle*. EMA/CHMP/CVMP/83833/2023.
- Urbina, F., Lentzos, F., Invernizzi, C., & Ekins, S. (2022). Dual use of artificial-intelligence-powered drug discovery. *Nature Machine Intelligence*, 4, 189-191. https://doi.org/10.1038/s42256-022-00465-9

See Chapter 1′s references for Mendez et al. (2019, ChEMBL); Chapter 5′s for the hERG QSAR methodology (Sanguinetti & Tristani-Firouzi, 2006) and Bemis & Murcko (1996, scaffold definition), both reused unchanged in Section 18.1; and Chapter 13′s for Veber et al. (2002), Baell & Holloway (2010, PAINS), and Bickerton et al. (2012, QED), reused unchanged for this chapter's own ADMET endpoint — all reused here rather than re-listed.

All dataset sizes, held-out evaluation metrics, and API responses cited in Section 18.1 were computed directly by running `train_model.py` and querying the real, running `service.py` on

2026-08-22, not taken from a secondary source — see `results/training_results.json` and `tests/test_deployment.py` to reproduce.